

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2025/2026

Università degli Studi di Torino,
Dipartimento di Informatica



Sistemi Esperti

Detti anche *Knowledge-Based Systems*

“A computer system that emulates the decision-making ability of a human expert in a restricted domain.”

[Giarratano & Riley 1998]

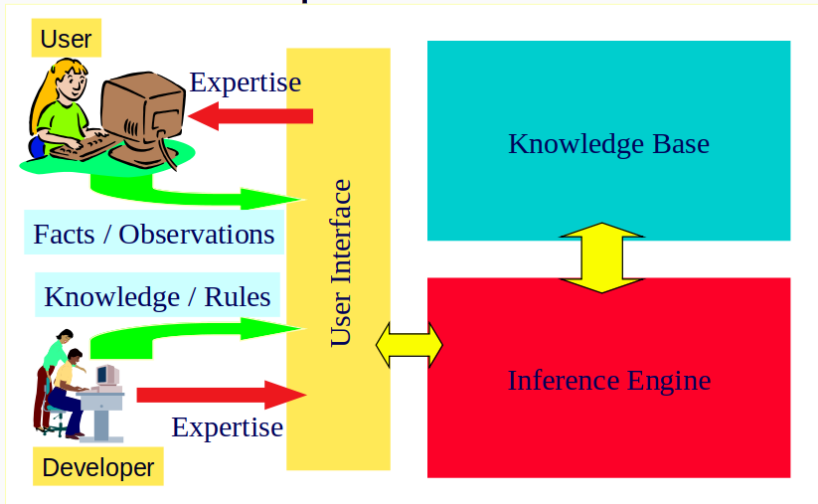
“An intelligent computer program that uses knowledge and inference procedures to solve problems that are difficult enough to require significant human expertise for their solutions.”

[Giarratano & Riley 1998]

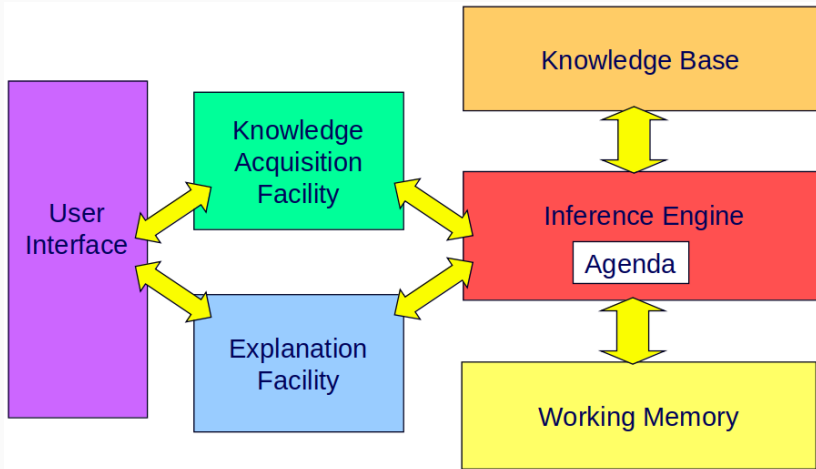
- Un sistema basato sulla conoscenza è un sistema in grado di risolvere problemi in un dominio limitato ma con prestazioni simili a quelle di un esperto umano del dominio stesso.
- Generalmente esamina un largo numero di possibilità e costruisce dinamicamente una soluzione.
- *“La potenza di un programma intelligente nel risolvere un problema dipende primariamente dalla quantità e qualità di conoscenza che possiede su tale problema”.* (Feigenbaum)

- Il programma non è un insieme di istruzioni immutabili che rappresentano la soluzione del problema, ma un ambiente in cui:
 - rappresentare;
 - utilizzare (*reasoning*);
 - modificareuna base di conoscenza.
- Caratterizzato dalle seguenti proprietà:
 - Specificità (conoscenza di dominio)
 - Rappresentazione esplicita della conoscenza.
 - Meccanismi di ragionamento.
 - Capacità di spiegazione.
 - Capacità di operare in domini poco strutturati

Cos'è un Sistema Esperto



Architettura di un Sistema esperto



Architettura di un Sistema esperto

- **Knowledge-Base [KB] (Rule-Base):** mantiene la conoscenza dell'esperto come regole condizione-azione (aka: if-then or premise-consequence)
- **Working Memory [WM]:** mantiene i fatti iniziali e quelli generati dalle inferenze
- **Inference Engine:**
 - *pattern matching*: confronta la parte if delle regole rispetto ai fatti della WM
 - regole che hanno la parte if soddisfatta sono dette **attivabili** e sono poste nell'Agenda
 - **Agenda**: elenco ordinato di regole attivabili, le regole sono ordinate in base alla loro priorità o in base ad altre strategie di preferenza/conflict resolution
 - **Execution**: la regola in cima all'agenda è selezionata ed eseguita (*firing*)
- **Explanation Facility:** fornisce una giustificazione delle soluzioni (*reasoning chain* $\equiv$ sequenza di regole che sono state attivate)
- **Knowledge Acquisition Facility:** aiuta l'ingegnere della conoscenza ad integrare nuove regole e a mantenerle nel tempo
- **User Interface:** consente all'utente di interagire con il sistema esperto (porre problemi e ricavarne una risposta)

Due direzioni di ragionamento

- **forward chaining** (data-driven)
 - ragionamento da fatti a conclusioni
 - non appena fatti (osservazioni) sono disponibili, questi sono usati per il matching degli antecedenti delle regole
 - spesso usati per real-time monitoring e controllo
 - CLIPS, OPS5

Due direzioni di ragionamento

- **forward chaining** (data-driven)
 - ragionamento da fatti a conclusioni
 - non appena fatti (osservazioni) sono disponibili, questi sono usati per il matching degli antecedenti delle regole
 - spesso usati per real-time monitoring e controllo
 - CLIPS, OPS5
- **backward chaining** (goal-driven)
 - partendo da un'ipotesi (goal), regole e fatti a supporto dell'ipotesi sono ricercati fino a quando tutte le parti dell'ipotesi non sono dimostrate
 - usata in sistemi diagnostici
 - MYCIN, PROLOG

Domini applicativi

- **Interpretazione:** Si analizzano dati complessi e potenzialmente affetti da rumore per la determinazione del loro significato (Dendral, Hearsay-II).
- **Diagnosi:** Si analizzano dati (con potenziale rumore) per la determinazione di malattie o errori (Mycin, ...).
- **Monitoring:** I dati si interpretano continuamente per la generazione di allarmi in situazioni critiche. Al sistema è richiesta una risposta in tempo reale soddisfacente (VM).
- **Planning e Scheduling:** Si determina una sequenza intelligente di azioni per raggiungere un determinato obiettivo (Molgen).
- **Previsione** (e.g., economica): Si desidera costruire un sistema in grado di prevedere il futuro in base a un appropriato modello del passato e del presente (Prospector).
- **Progetto e configurazione:** Il Sistema Esperto deve essere in grado di progettare sistemi partendo da ben determinate specifiche (R1/XCON).

Quando non usare un sistema esperto

- Sistemi esperti non sono da preferire quando:
 - esistono algoritmi "tradizionali" efficienti per risolvere il problema considerato
 - l'aspetto principale del problema è la computazione, e non la conoscenza
 - la conoscenza non può essere modellata o elicitata in modo efficiente
 - l'utente finale è riluttante ad applicare un sistema esperto per via della criticità del task (e.g., somministrare medicinali, pilotare aerei,...)
- Posso risolvere il problema di pianificazione con un sistema esperto?

Quando non usare un sistema esperto

- Sistemi esperti non sono da preferire quando:
 - esistono algoritmi "tradizionali" efficienti per risolvere il problema considerato
 - l'aspetto principale del problema è la computazione, e non la conoscenza
 - la conoscenza non può essere modellata o elicitata in modo efficiente
 - l'utente finale è riluttante ad applicare un sistema esperto per via della criticità del task (e.g., somministrare medicinali, pilotare aerei,...)
- Posso risolvere il problema di pianificazione con un sistema esperto?
 Si! (Ma con molta fatica)

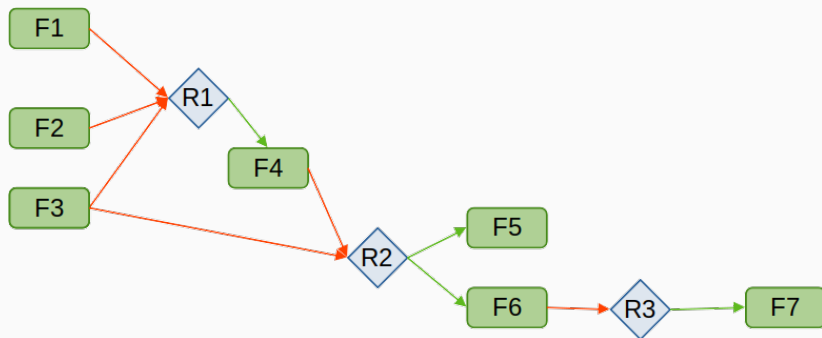
Un sistema a regole di produzione ha la stessa potenza espressiva di una macchina di Turing, quindi può risolvere qualsiasi problema risolvibile con un algoritmo tradizionale, ma non sempre è il formalismo più adeguato.

CLIPS

- *'C' Language Integrated Production System (CLIPS)*
- Sviluppato dalla NASA tra il 1985 e il 1996 (tutt'ora mantenuto)
- E' uno strumento per realizzare sistemi esperti basati su *regole*
- E' lo strumento ideale quando un problema complesso può essere risolto sfruttando conoscenza esplicita (anche euristica, derivata cioè dall'esperienza)
- Un sistema come CLIPS è detto **eterarchico**, cioè “piatto”

- Il motore inferenziale è **forward chaining**
 1. Determina tutte le regole *attivabili* (antecedente soddisfatto)
 2. Ordina le regole attivabili in *agenda*
 3. Esegue (*fire*) la regola in cima all'agenda, con la conseguente modifica dei fatti in WM
 4. Torna al passo 1 e ripeti fino a quando non ci sono regole attivabili
- CLIPS produce un risultato per mezzo di una concatenazione di regole
- La concatenazione stessa è una spiegazione delle conclusioni perché dimostra come si è giunti ad una certa conclusione

Ciclo di esecuzione di CLIPS: Rule Chain



CLIPS Primitive Data Types

- float
- integer
- **symbol**: e.g. this-is-a-symbol, wrzlbrmft, !?@*+
- string: e.g. "This is a string"
- external address (indirizzi o strutture dati restituite da funzioni user-defined)
- instance name (Cool)
- instance address (Cool)

- Avviare CLIPS da riga di comando (UNIX):
\$./clips

- Avviare CLIPS da riga di comando (UNIX):

```
$ ./clips
```

appare il prompt

```
CLIPS>
```

- Avviare CLIPS da riga di comando (UNIX):

```
$ ./clips
```

appare il prompt

```
CLIPS>
```

- Terminare CLIPS

```
CLIPS> (exit)
```

- Avviare CLIPS da riga di comando (UNIX):

```
$ ./clips
```

appare il prompt

```
CLIPS>
```

- Terminare CLIPS

```
CLIPS> (exit)
```

NB. CLIPS è un interprete di regole, quindi possiamo modificare la WM e la KB in qualsiasi momento aggiungendo/rimuovendo fatti e regole.

- "chunk of information" elemento indivisibile di informazione
- sono definiti da un nome di relazione
- possono avere zero o più *slot* (i.e., *facet*)
(se non hanno slot il nome della relazione li identifica univocamente)
- costruito `deftemplate` per definire fatti strutturati (detti non ordinati)
- costruito `def facts` per definire il set iniziale di fatti

- **fatto ordinato** (no slot)
 (person-name Franco L Verdi)
 (person-name Verdi Franco L)
- **fatto non ordinato** (esempio di **template**)
 (deftemplate person "commento opzionale"
 (slot name)
 (slot age)
 (slot eye-color)
 (slot hair-color))
- **fatto non ordinato** (esempio di **istanza** di template)
 (person (name "Franco L. Verdi")
 (age 46)
 (eye-color brown)
 (hair-color brown))

Modellare la conoscenza con i templates

```
(deftemplate student "a student record"
  (slot name (type STRING))
  (slot age (type INTEGER) (default 18) (range 0 ?VARIABLE) )
  (slot gender (type SYMBOL) (allowed-symbols male female))
)
```

```
(deftemplate volleyball-team
  (slot name (type STRING))
  (multislot player (type STRING) (cardinality 6 6))
  (multislot alternates (type STRING) (cardinality 0 2)) )
```

Modellare la conoscenza con i templates

```
(deftemplate student "a student record"
  (slot name (type STRING))
  (slot age (type INTEGER) (default 18) (range 0 ?VARIABLE) )
  (slot gender (type SYMBOL) (allowed-symbols male female))
)
```

```
(deftemplate volleyball-team
  (slot name (type STRING))
  (multislot player (type STRING) (cardinality 6 6))
  (multislot alternates (type STRING) (cardinality 0 2)) )
```

?VARIABLE indica che non c'è un limite superiore specificato

Modellare la conoscenza con i templates

```
(deftemplate student "a student record"
  (slot name (type STRING))
  (slot age (type INTEGER) (default 18) (range 0 ?VARIABLE) )
  (slot gender (type SYMBOL) (allowed-symbols male female))
)
```

```
(deftemplate volleyball-team
  (slot name (type STRING))
  (multislot player (type STRING) (cardinality 6 6))
  (multislot alternates (type STRING) (cardinality 0 2)) )
```

male e female sono gli unici valori ammessi

Modellare la conoscenza con i templates

```
(deftemplate student "a student record"
  (slot name (type STRING))
  (slot age (type INTEGER) (default 18) (range 0 ?VARIABLE) )
  (slot gender (type SYMBOL) (allowed-symbols male female))
)
```

```
(deftemplate volleyball-team
  (slot name (type STRING))
  (multislot player (type STRING) (cardinality 6 6))
  (multislot alternates (type STRING) (cardinality 0 2)) )
```

cardinalità minima e massima del multislot: quanti valori deve almeno e può al più contenere

Esempio di deffacts

il costrutto `deffacts` è usato unicamente per stabilire quali fatti sono veri all'avvio del motore inferenziale

```
(deffacts famiglia-verdi "alcuni membri della famiglia Verdi"
  (person (name "Luigi") (age 46) (eye-color brown) ( hair-color brown))
  (person (name "Maria") (age 40) (eye-color blue) ( hair-color brown))
  (person (name "Marco") (age 25) (eye-color brown) ( hair-color brown))
  (person (name "Lisa") (age 20) (eye-color blue) ( hair-color blonde))
)
```

- I fatti sono effettivamente asseriti in WM solo dopo aver dato il comando `(reset)`
- Per elencare i fatti presenti in memoria si usa `(facts)`

Manipolazione di fatti

- aggiungere fatti
`(assert <fact>+)`
- rimuovere fatti
`(retract <fact-index>+)`
- modificare fatti
`(modify <fact-index> (<slot-name> <slot-value>)+ )`
equivale a rimuovere il fatto originale e aggiungere il nuovo fatto modificato con un indice incrementato
- duplicare fatti
`(duplicate <fact-index> (<slot-name> <slot-value>)+ )`
- ispezionare la working memory
`(facts)` stampa la lista di fatti
`(watch facts)` automaticamente mostra i cambiamenti che occorrono nella WM a seguito dell'esecuzione delle regole
- `(unwatch facts)` per disabilitare il watching sui fatti

```
(defrule <rule name> ["comment"]  
  <patterns>* ; left-hand side (LHS)  
              ; or antecedent of the rule  
=>  
  <actions>*) ; right-hand side (RHS)  
              ; or consequent of the rule
```

```
(defrule birthday-FLV
  (person (name "Luigi")
    (age 46)
    (eye-color brown)
    (hair-color brown))
  (date-today April-13-02)
=>
  (printout t "Happy birthday, Luigi!")
  (assert (there-s-a-party))
)
```

NB. Questo è un caso molto particolare di regola perché i pattern dell'antecedente sono completamente istanziati, in generale si ricorre all'uso di **variabili**

Regole - ispezionare e tracciare

- (rules) elenca le regole definite fino a questo momento nella KB
- (ppdefrule *nome-regola*) mostra la definizione della regola
- (agenda) mostra l'agenda attuale: elenco ordinato di regole attivabili
 - la regola in cima sarà la prossima ad essere eseguita
 - per ogni regola sono indicati i fatti che l'attivano
- (watch rules) mostra, durante l'esecuzione, quali regole sono eseguite
- (watch activations) mostra, durante l'esecuzione, quali attivazioni hanno permesso l'esecuzione delle regole
- (unwatch rules) e (unwatch activations) per disattivare il watching

Debugging... a sort of

- Per evitare di avere le informazioni di watching direttamente nell'interfaccia dell'interprete, si può usare il comando
`(dribble-on filename)`
che salva tutto ciò che viene visualizzato sul terminale sul file indicato, compreso gli input dell'utente.
`(dribble-off)`
Disabilita il comando.
- Breakpoints:
`(set-break <rulename>)`
`(remove-break <rulename>)`
`(show-breaks)`

- Nomi simbolici che cominciano con '?'
- **binding:**
 - variabili in un pattern (LHS) sono legate a valori di fatti in WM
 - ogni occorrenza di una variabile in una regola ha lo stesso valore
 - l'occorrenza più a sinistra (la prima) determina il valore
 - il binding è valido solo localmente ad una regola
 - variabili sono usate anche come maniglie (handlers) per accedere ai fatti della WM es. `?age <- (age harry 17)`
- wildcards (~ variabili senza nome)
 - ? match con qualsiasi valore in un campo singolo (slot) di un fatto
 - \$? match con zero o più valori in un multislots di un fatto

Esempio Modify Fact

```
(defrule birthday-FLV
  ?person <- (person (name "Luigi")
    (age 46)
    (eye-color brown)
    (hair-color brown))
  (date-today April-13-02)
=>
  (printout t "Happy birthday, Luigi!")
  (modify ?person (age 47))
)
```

```
(defrule find-blue-eyes
  (person (name ?name)(eye-color blue))
=>
  (printout t ?name " has blue eyes." crlf))
```

La **regola scatta per tutte le persone che hanno gli occhi blu**. I valori degli slot name, age, ecc. non contano, ma con ?name siamo in grado di recuperare il nome della specifica persona che attiva la regola. Però non vorrei entrare in un loop infinito in cui la regola stampa all'infinito *"Lisa has blue eyes"*.

Altro Esempio

```
(defrule find-blue-eyes
  (person (name ?name)(eye-color blue))
=>
  (printout t ?name " has blue eyes." crlf))
```

La **regola scatta per tutte le persone che hanno gli occhi blu**. I valori degli slot name, age, ecc. non contano, ma con ?name siamo in grado di recuperare il nome della specifica persona che attiva la regola

Rifrazione

è un meccanismo alla base del patter-matching che impedisce di attivare due volte una regola sugli stessi fatti

La **regola scatta una sola volta per ogni persona con gli occhi blu**.

"Lisa has blue eyes" viene stampato una volta sola

Avviare il motore inferenziale di CLIPS

- Supponiamo che il file `today.clp` abbia il seguente contenuto

```
(defrule start
  (greet) => (printout t "hello" crlf)
)
(deffacts MyFacts
  (greet)
)
```

Avviare il motore inferenziale di CLIPS

- Supponiamo che il file `today.clp` abbia il seguente contenuto

```
(defrule start
  (greet) => (printout t "hello" crlf)
)
(deffacts MyFacts
  (greet)
)
```

- Possiamo caricare il contenuto di un file con il comando `load`

```
CLIPS> (load today.clp)
CLIPS> (facts)
f-0 (initial-fact)
```

Avviare il motore inferenziale di CLIPS

- Supponiamo che il file `today.clp` abbia il seguente contenuto

```
(defrule start
  (greet) => (printout t "hello" crlf)
)
(deffacts MyFacts
  (greet)
)
```

- Possiamo caricare il contenuto di un file con il comando `load`

```
CLIPS> (load today.clp)
```

```
CLIPS> (facts)
```

```
f-0 (initial-fact) (è un fatto di default utilizzato dal motore)
```

Avviare il motore inferenziale di CLIPS

- Supponiamo che il file `today.clp` abbia il seguente contenuto

```
(defrule start
  (greet) => (printout t "hello" crlf)
)
(deffacts MyFacts
  (greet)
)
```

- Possiamo caricare il contenuto di un file con il comando `load`

```
CLIPS> (load today.clp)
CLIPS> (facts)
f-0 (initial-fact)
CLIPS> (reset) (carica i fatti nei costrutti deffacts)
CLIPS> (facts)
f-0 (initial-fact)
f-1 (greet)
```

Avviare il motore inferenziale di CLIPS

- A questo punto possiamo avviare il motore inferenziale con il comando `run`

```
CLIPS> (run)
```

```
CLIPS>hello
```

Avviare il motore inferenziale di CLIPS

- A questo punto possiamo avviare il motore inferenziale con il comando `run`

```
CLIPS> (run)
```

```
CLIPS>hello
```

- `(run)` fa sì che il motore inferenziale esegua una catena di attivazioni di regole fino a quando:
 - il comando `(halt)` compare nel conseguente di una regola
 - non ci sono più regole attivabili (agenda è vuota)
- `(run n)` dove *n* è un intero che indica il numero massimo di regole che si vogliono eseguire, poi il motore si ferma anche se l'agenda non è vuota

Esercitazione

Usare CLIPS per rappresentare predicati e formule logiche

- Vogliamo scrivere una piccola KB che ci permetta di inferire le parentele tra un gruppo di persone, ad esempio vorremmo dire che

- se x è un genitore di y , allora è anche un antenato di y

$$\forall x, y, \text{parent}(x, y) \rightarrow \text{ancestor}(x, y)$$

- se x è un genitore di un antenato z di y , allora anche x è un antenato di y

$$\forall x, y, z, \text{parent}(x, z) \wedge \text{ancestor}(z, y) \rightarrow \text{ancestor}(x, y)$$

- essere padre/madre vuol dire essere un genitore

$$\forall x, y, \text{father}(x, y) \rightarrow \text{parent}(x, y)$$

$$\forall x, y, \text{mother}(x, y) \rightarrow \text{parent}(x, y)$$

- avere figli, essere umani e maschi vuol dire essere padri

$$\forall x, y, \text{hasChild}(x, y) \wedge \text{human}(x) \wedge \text{male}(x) \rightarrow \text{father}(x)$$

- analogamente

$$\forall x, y, \text{hasChild}(x, y) \wedge \text{human}(x) \wedge \text{female}(x) \rightarrow \text{mother}(x)$$

Usare CLIPS per rappresentare formule logiche

- Supponiamo di essere a conoscenza delle seguenti informazioni:

```
human(Luigi), male(Luigi)
human(Marta), female(Marta)
human(Luca), male(Luca),
human(Maria), female(Maria),
human(Lucrezia), female(Lucrezia),
human(Ludovico), male(Ludovico),
human(Miriam), female(Miriam),
hasChild(Luigi, Marta)
hasChild(Luigi, Luca)
hasChild(Marta, Maria)
hasChild(Marta, Lucrezia)
hasChild(Maria, Ludovico)
hasChild(Ludovico, Miriam)
```

- Come possiamo codificare questa conoscenza in CLIPS?

Esercizio 1

- Scrivere un programma CLIPS che deduca tutte le possibili relazioni di parentela
- **NB** questa è una forma di inferenza monotona senza backtracking
- Modificare poi il codice in modo tale da catturare quest'ulteriore implicazione:

$$\forall x, y, z, \text{parent}(x, y) \wedge \text{parent}(x, z) \rightarrow \text{sibling}(y, z)$$

Avendo cura che il fatto *sibling* venga aggiunto una sola volta in WM (i.e., $\text{sibling}(y, z) \equiv \text{sibling}(z, y)$)

Esercizio 2

- Scrivere un programma CLIPS che simuli una ricerca backward
- partendo da un goal della forma
 `(maingoal ancestor Luigi Miriam)`
 restituisca YES se può dimostrare che Luigi è un antenato di Miriam
- in questo caso la ricerca deve essere guidata dal goal
- non tutte le possibili relazioni parentali saranno inferite

Pattern Matching Avanzato

Field Constraints

Vincoli definiti per il singolo campo usati per filtrare il pattern matching

- **& and**: il valore del campo deve soddisfare una congiunzione di vincoli

`(name ?name & Harvey)`

`(name ?name &~Simpson)`

- **~ not**: il campo può prendere qualsiasi valore tranne quello specificato

`(name ~Simpson)`

`(age ~40)`

- **| or**: sono specificati valori alternativi ammissibili per uno stesso campo

`(name Simpson|Oswald)`

`(age 30|40|50)`

- **: expression**: aggiungi l'espressione che segue come vincolo

`(age ?age&:(> ?age 20))`

`(name ?name&:(eq (sub-string 1 1 ?name) "S"))`

Field Constraints - Esempio

```
(defrule silly-eye-hair-match
  (person (name ?name1)
    (eye-color ?eyes1 & blue|green)
    (hair-color ?hair1 & ~black))
  (person (name ?name2 & ~?name1)
    (eye-color ?eyes2 & ~?eyes1)
    (hair-color ?hair2 & red|?hair1))
=>
  (printout t ?name1 " has "?eyes1 " eyes and " ?hair1 " hair." crlf)
  (printout t ?name2 " has "?eyes2 " eyes and " ?hair2 " hair." crlf)
)
```

Field Constraints vs test

- In alternativa ai field constraint è possibile usare la clausola
(test *boolean-function*)

- Ad esempio

```
(defrule silly-eye-hair-match
  (person (name ?name1)
    (eye-color ?eyes1)
    (hair-color ?hair1))
  (person (name ?name2)
    (eye-color ?eyes2)
    (hair-color ?hair2 & red|?hair1))
  (test (or (eq ?eyes1 blue) (eq ?eyes1 green) ) )
  (test (neq ?hair1 black))
  (test (neq ?name1 ?name2))
  (test (neq ?eyes1 ?eyes2))
  (test (or (eq ?hair2 red) (eq ?hair2 ?hair1) ))
  =>
  (printout t ?name1 " has "?eyes1 " eyes and " ?hair1 " hair." crlf)
  (printout t ?name2 " has "?eyes2 " eyes and " ?hair2 " hair." crlf)
)
```

Field Constraints vs test

- In alternativa ai field constraint è possibile usare la clausola
(test *boolean-function*)

- Ad esempio

```
(defrule silly-eye-hair-match
  (person (name ?name1)
    (eye-color ?eyes1)
    (hair-color ?hair1))
  (person (name ?name2)
    (eye-color ?eyes2)
    (hair-color ?hair2 & red|?hair1))
  (test (or (eq ?eyes1 blue) (eq ?eyes1 green) ) )
  (test (neq ?hair1 black))
  (test (neq ?name1 ?name2))
  (test (neq ?eyes1 ?eyes2))
  (test (or (eq ?hair2 red) (eq ?hair2 ?hair1) ))
=>
  (printout t ?name1 " has "?eyes1 " eyes and " ?hair1 " hair." crlf)
  (printout t ?name2 " has "?eyes2 " eyes and " ?hair2 " hair." crlf)
)
```

C'è un evidente vantaggio ad usare i field constraint, quale?

Pattern Matching con Multislot

- I multislot sono alla base della definizione dei fatti ordinati, infatti, quando definiamo un fatto ordinato:
`(data 1.0 blue "red")`
è come se stessimo creando un fatto non ordinato il cui template è
`(deftemplate data (multislot values))`

Pattern Matching con Multislot

- I multislot sono alla base della definizione dei fatti ordinati, infatti, quando definiamo un fatto ordinato:
`(data 1.0 blue "red")`
è come se stessimo creando un fatto non ordinato il cui template è
`(deftemplate data (multislot values))`
- In altri termini, quando definiamo un fatto ordinato è come se stessimo ricorrendo alla struttura (*<nome-relazione>*
<multislot-valori>)

Pattern Matching con Multislot

- I multislot sono alla base della definizione dei fatti ordinati, infatti, quando definiamo un fatto ordinato:
`(data 1.0 blue "red")`
è come se stessimo creando un fatto non ordinato il cui template è
`(deftemplate data (multislot values))`
- In altri termini, quando definiamo un fatto ordinato è come se stessimo ricorrendo alla struttura (*<nome-relazione>*
<multislot-valori>)
- poiché un multislot può anche essere vuoto, riusciamo a catturare anche i casi in cui un fatto ordinato è identificato dal solo nome di relazione. Es.:
`(data)`

Pattern Matching con Multislot

- I multislot sono alla base della definizione dei fatti ordinati, infatti, quando definiamo un fatto ordinato:
`(data 1.0 blue "red")`
è come se stessimo creando un fatto non ordinato il cui template è
`(deftemplate data (multislot values))`
- In altri termini, quando definiamo un fatto ordinato è come se stessimo ricorrendo alla struttura (*<nome-relazione>*
<multislot-valori>)
- poiché un multislot può anche essere vuoto, riusciamo a catturare anche i casi in cui un fatto ordinato è identificato dal solo nome di relazione. Es.:
`(data)`
- È importante ricordare questa caratteristica, perché spiega come mai possiamo usare il simbolo `$?` per il pattern matching e il binding di variabili multislot sia per i fatti ordinati sia per quelli non ordinati

- Dati i seguenti fatti

```
(deffacts data-facts
  (data 1.0 blue "red")
  (data 1 blue)
  (data 1 blue red)
  (data 1 blue RED)
  (data 1 blue red 6.9)
  (data 2 red))
```

- il pattern (data \$?multi) potrà essere unificato con ciascuno dei fatti indicati: di volta in volta (data \$?multi) assumerà come valore la lista dei simboli/stringhe/numeri che seguono la relazione data

- Dati i seguenti fatti

```
(deffacts data-facts  
  (data 1.0 blue "red")  
  (data 1 blue)  
  (data 1 blue red)  
  (data 1 blue RED)  
  (data 1 blue red 6.9)  
  (data 2 red))
```

- dato il pattern (data ? blue red \$?) con quali fatti potrà essere unificato?

- Dati i seguenti fatti

```
(deffacts data-facts
```

```
(data 1.0 blue "red")
```

```
(data 1 blue)
```

```
(data 1 blue red)
```

```
(data 1 red blue)
```

```
(data 1 blue red 6.9)
```

```
(data 2 red blue 2.0))
```

- dato il pattern (data \$?prima blue \$?dopo) con quali fatti potrà essere unificato?

- Dati i seguenti fatti

```
(deffacts data-facts  
  (data 1.0 blue "red")  
  (data 1 blue)  
  (data 1 blue red)  
  (data 1 red blue)  
  (data 1 blue red 6.9)  
  (data 2 red blue 2.0))
```

- dato il pattern `(data ?x $?y ?z)` con quali fatti potrà essere unificato?

Pattern Matching con Multislot - funzioni predefinite

```
(deftemplate box (slot id) (multislot cont))  
(deffacts init  
  (box (id A) (cont a1 a1 a3))  
  (box (id B) (cont b1 b2))  
  (box (id C))  
  (removable b1)  
  (removable b2)  
  ...)
```

- Stabilire il numero di elementi in un multislot

```
(defrule check_void (declare (salience 100))  
  (box (id ?id) (cont $?x))  
  (test (eq (length$ $?x) 0))  
=> (printout t "the box " ?id " is void " crlf))
```

Pattern Matching con Multislot - funzioni predefinite

```
(deftemplate box (slot id) (multislot cont))  
(deffacts init  
  (box (id A) (cont a1 a1 a3))  
  (box (id B) (cont b1 b2))  
  (box (id C))  
  (removable b1)  
  (removable b2)  
  ...)
```

- Stabilire l'appartenenza a un multislot e rimozione da un multislot

```
(defrule remove  
  ?f1 <- (removable ?r)  
  ?f <- (box (id B) (cont $?x))  
  (test (member$ ?r $?x))  
  => (modify ?f (cont (delete-member$ $?x ?r))))
```

Pattern Matching con Multislot - funzioni predefinite

```
(deftemplate box (slot id) (multislot cont))  
(deffacts init  
  (box (id A) (cont a1 a1 a3))  
  (box (id B) (cont b1 b2))  
  (box (id C))  
  (loadable-first p1)  
  (loadable-first p2)  
  ...)
```

- Aggiungere in una posizione precisa in un multislot
 (defrule add-first (declare (salience 12))
 ?f2 <- (box (id A) (cont \$?x))
 ?f1 <- (loadable-first ?l1)
 => (modify ?f2 (cont (insert\$ \$?x 1 ?l1)))
 (retract ?f1))

```
(deftemplate box (slot id) (multislot cont))  
(deffacts init  
  (box (id A) (cont a1 a1 a3))  
  (box (id B) (cont b1 b2))  
  (box (id C))  
  (loadable-second s1)  
  ...)
```

- Aggiungere in una posizione precisa in un multislot

```
(defrule add-second (declare (salience 11))  
  ?f2 <- box (id A) (cont $?x))  
  ?f1 <- (loadable-second ?l1)  
=> (modify ?f2 (cont (insert$ $?x 2 ?l1)))  
    (retract ?f1))
```

Pattern Matching con Multislot - funzioni predefinite

```
(deftemplate box (slot id) (multislot cont))  
(deffacts init  
  (box (id A) (cont a1 a1 a3))  
  (box (id B) (cont b1 b2))  
  (box (id C))  
  (loadable-last l1)  
  ...)
```

- Aggiungere in una posizione precisa in un multislot

```
(defrule add-last (declare (salience 10))  
  ?f2 <- (box (id C) (cont $?x))  
  ?f1 <- (loadable-last ?l1)  
=> (modify ?f2 (cont (insert$ $?x (+ (length$ $?x) 1) ?l1)))  
  (retract ?f1))
```

- Richiesta del Primo Elemento di un Multifield

`(first$ <multifield-expression>)`

Esempio

`CLIPS> (first$ (create$ a b c))`

`(a)`

`CLIPS> (first$ (create$))`

`()`

- Restituzioni di tutti gli elementi di un Multifield tranne il primo
(rest\$ <multifield-expression>)

Esempio

```
CLIPS> (rest$ (create$ a b c))
```

```
(b c)
```

```
CLIPS> (rest$ (create$))
```

```
()
```

- Rimpiazzamento di campi in un Multifield

```
(replace$ <multifield-expression>  
      <begin-integer-expression> <end-integer-expression>  
      <single-or-multi-field-expression>+)
```

Esempio

```
CLIPS> (replace$ (create$ drill wrench pliers) 3 3 machete)  
(drill wrench machete)  
CLIPS> (replace$ (create$ a b c d) 2 3 x y (create$ q r s))  
(a x y q r s d)
```

```
(deftemplate box (slot id) (multislot cont))  
(defacts init  
(box (id A) (cont a1 a1 a3))  
...)
```

- Ennesimo elemento di un multislot

```
(defrule test-nth (declare (salience 100))  
  ?f2 <- (box (id ?id) (cont $?x))  
  (test (eq (nth$ 2 $?x) a1))  
=> (printout t " " ?id "has in position 2 a1" crlf))
```

Pattern Matching con Multislot - funzioni predefinite

- Rimuovere elementi da un multislot

```
(delete$ <multifield-expression>  
      <begin-integer-expression> <end-integer-expression>)
```

Esempio

```
CLIPS> (delete$ (create$ hammer drill saw pliers wrench) 3 4)  
(hammer drill wrench)  
CLIPS> (delete$ (create$ computer printer hard-disk) 1 1)  
(printer hard-disk)
```

- Creazione di un Multifield da Stringhe

`(explode$ <string-expression>)`

Esempio

```
CLIPS> (explode$ "hammer drill saw screw")
```

```
(hammer drill saw screw)
```

```
CLIPS> (explode$ "1 2 abc 3 4 \"abc\" \"def\"")
```

```
(1 2 abc 3 4 "abc" "def")
```

```
CLIPS> (explode$ "?x ~ ")
```

```
("?x" "~" " ")
```

- Verifica se un multifield è sottoinsieme di un altro multifield
(subsetp <multifield-expression> <multifield-expression>)

Esempio

```
CLIPS> (subsetp (create$ hammer saw drill)
(create$ hammer drill wrench pliers saw))
TRUE
```

```
CLIPS> (subsetp (create$ wrench crowbar)
(create$ hammer drill wrench pliers saw))
FALSE
```

- Estrazione di una sottosequenza da un Multifield

```
(subseq$ <multifield-value>  
<begin-integer-expression> <end-integer-expression>)
```

Esempio

```
CLIPS> (subseq$ (create$ hammer drill wrench pliers) 3 4)  
(wrench pliers)
```

```
CLIPS> (subseq$ (create$ 1 "abc" def "ghi" 2) 1 1)  
(1)
```

Strategie di Conflict Resolution

- **agenda:**
 - lista di tutte le regole che hanno la parte sinistra (LHS) soddisfatta (e non sono ancora state eseguite)
 - ogni modulo (v. seguito) ha la sua propria agenda
 - agisce come uno stack
 - la regola al top è la prima ad essere eseguita
- Come sono ordinate le regole nell'agenda?

- La posizione di una **regola attiva** nell'agenda è determinata come segue:
 - a) in base alla *salience* (priorità):
 - tutte le regole con *salience* più bassa sono sotto la regola corrente;
 - tutte le regole con *salience* più alta sono sopra la regola corrente
 - b) a parità di *salience*, è la strategia di risoluzione del conflitto (*conflict resolution strategy*) a determinare la posizione;
 - c) se una regola è attivata (insieme ad altre) dagli stessi fatti WM, e i passi a) e b) non sono in grado di specificare l'ordine, allora l'ordine con cui le regole sono scritte nel sorgente stabilisce la priorità (in alcuni casi si segue lo stesso ordine con cui sono scritte le regole, in altri si usa l'ordine inverso)

- **salience:**

- L'ingegnere della conoscenza può assegnare una priorità ad una regola tramite la proprietà *salience* delle regole.
- I valori di salience cadono nell'intervallo $[-10000, 10000]$.
- **NB** In genere non è mai buona norma attribuire salience in modo da forzare un ordine preciso di attivazione delle regole.

```
(defrule test-1 (declare (salience 99))  
  (fire test-1)  
=>  
  (printout t "Rule test-1 firing." crlf))
```

- **Depth:** regole attive più recenti in cima alle altre regole di pari salience
- **Breadth:** regole attive più recenti dopo regole di pari salience
- **Simplicity:** regole attive più recenti sopra a regole con uguale o più alta *specificità*
- **Complexity:** regole attive più recenti sopra a regole con uguale o più bassa specificità
- **LEX, MEA:** da *OPS5*
- **Random:** alle nuove regole attive viene assegnato un valore casuale con cui sono poi ordinate in agenda

Strategie di conflict resolution: Depth

- sono preferite regole attivate dai fatti più recenti
- esempio:
 - il fatto-a attiva la regola-1 e la regola-2
 - il fatto-b attiva la regola-3 e la regola-4
 - se viene asserito il fatto-a prima del fatto-b:
 - la regola-3 e la regola-4 saranno poste al di sopra della regola-1 e della regola-2 nell'agenda
 - la posizione della regola-1 relativa alla regola-2 e della regola-3 relativa alla regola-4 dipenderà dall'ordine con cui sono scritte le regole nel file sorgente

- sono preferite regole attivate dai fatti meno recenti
- esempio:
 - il fatto-a attiva la regola-1 e la regola-2
 - il fatto-b attiva la regola-3 e la regola-4
 - se viene asserito il fatto-a prima del fatto-b:
 - la regola-1 e la regola-2 saranno poste al di sopra della regola-3 e della regola-4 nell'agenda
 - la posizione della regola-1 relativa alla regola-2 e della regola-3 relativa alla regola-4 dipenderà dall'ordine con cui sono scritte le regole nel file sorgente

- **Specificity:** La specificity di una regola è determinata dal numero di confronti e binding effettuati nella LHS della regola.
- **Simplicity Strategy**
 - Fra tutte le regole con lo stesso salience, le nuove regole attivate sono posizionate al di sopra di tutte le attivazioni di regole con maggiore o uguale specificità.
- **Complexity Strategy**
 - Fra tutte le regole con stesso salience, le nuove regole attivate sono poste al di sopra di tutte le attivazioni di regole con minore o uguale specificità.

Strategie di conflict resolution: LEX

- A parità di salience, le regole attivate sono ordinate come segue:
 - Ogni fatto è etichettato con un “time tag” per indicarne la recentezza rispetto agli altri fatti
 - Le attivazioni sono ordinate per “recentezza” cioè dal fatto più recente a quello meno recente

Esempio di attivazioni di regole

attivazioni nell'ordine con cui sono definiti

rule1: f-1, f-2, f-3

rule2: f-2, f-1

rule3: f-2, f-1

rule4: f-1, f-4

come sono ordinati con LEX

rule4: f-4, f-1

rule1: f-3, f-2, f-1

rule3: f-2, f-1

rule2: f-2, f-1

- - Le attivazioni sono confrontate rispetto alla recentezza del fatto che soddisfa il primo pattern delle regole (cioè le attivazioni sono ordinate come nel caso precedente)
 - A parità di time tag del primo fatto si confrontano i time tag dei fatti successivi

Esempio di attivazioni di regole

pattern nell'ordine con cui sono definiti

rule1: f-1, f-2, f-3

rule2: f-2, f-1

rule3: f-2, f-1

rule4: f-1, f-4

come sono ordinati con LEX

rule3: f-2, f-1

rule2: f-2, f-1

rule4: f-1, f-4

rule1: f-1, f-2, f-3

Strategie di conflict resolution: set-strategy

- Per impostare una strategia di conflict resolution è sufficiente utilizzare il comando (`set-strategy <nome-strategia>`)

Strategie di conflict resolution: set-strategy

- Per impostare una strategia di conflict resolution è sufficiente utilizzare il comando `(set-strategy <nome-strategia>)`
- ad esempio:
`CLIPS>(set-strategy breadth)`
`depth ->` CLIPS stampa la vecchia strategia come risultato

Strategie di conflict resolution: set-strategy

- Per impostare una strategia di conflict resolution è sufficiente utilizzare il comando `(set-strategy <nome-strategia>)`
- ad esempio:
`CLIPS>(set-strategy breadth)`
`depth ->` CLIPS stampa la vecchia strategia come risultato
- è un comando come tutti gli altri e può comparire nel conseguente delle regole; ad esempio:

```
(defrule change-to-breadth
  (change-b)
=>
  (set-strategy breadth)
)
```

Strategie di conflict resolution: set-strategy

- Per impostare una strategia di conflict resolution è sufficiente utilizzare il comando `(set-strategy <nome-strategia>)`
- ad esempio:
`CLIPS>(set-strategy breadth)`
`depth ->` CLIPS stampa la vecchia strategia come risultato
- è un comando come tutti gli altri e può comparire nel conseguente delle regole; ad esempio:

```
(defrule change-to-breadth
  (change-b)
=>
  (set-strategy breadth)
)
```
- per conoscere la strategia corrente si può usare il comando `(get-strategy)`

Espressioni Antecedenti Complesse

- Per default i pattern nell'antecedente delle regole sono considerati in congiunzione (*and* impliciti) ma non è l'unica possibilità

- Per default i pattern nell'antecedente delle regole sono considerati in congiunzione (*and* impliciti) ma non è l'unica possibilità

- pattern in or:

```
(or (pattern1) (pattern2))
```

soddisfatta se almeno uno dei due pattern unifica con i fatti della WM

```
(defrule celebrate
```

```
(or (birthday) (anniversary))
```

```
=>
```

```
(printout t "Let's have a party!" crlf))
```

- not:

`(not (pattern))`

soddisfatto quando nessun fatto unifica con il pattern

```
(defrule working-day
```

```
  (not (birthday))
```

```
  (not (anniversary))
```

`=>`

```
  (printout t "Let's work!" crlf))
```

- exists:

`(exists (pattern))`

soddisfatto per un unico fatto che unifica

`(defrule emergency-report`

`(exists`

`(or (emergency (emergency-type fire))`

`(emergency (emergency-type bomb)))`

`)`

`=>`

`(printout t \There is an emergency.\ crlf )`

`)`

Se anche la WM contenesse entrambe le emergenze, la regola scatterebbe una volta sola

Regole e Connettivi Logici

- forall:

(forall (pattern))

soddisfatto se pattern vale per tutti i fatti che unificano.

Tipicamente pattern è una condizione complessa, altrimenti ha poco significato.

```
(defrule evacuated-all-buildings
  (forall (emergency (emergency-type fire | bomb)
              (location ?building) )
    (evacuated (building ?building))
  )
```

=>

```
(printout t \All buildings with emergency are evacuated \ cr
```

La regola scatta solo se per tutti i ?building dove è stata riscontrata una emergenza è anche stato dato l'ordine di evacuazione.

- forall:

Situazione 1: la regola scatta (una volta sola)

```
(emergency (emergency-time fire) (location library))  
(evacuated (building library))  
(emergency (emergency-time bomb) (location main-room))  
(evacuated (building main-room))
```

Situazione 2: la regola NON scatta

```
(emergency (emergency-time fire) (location library))  
(evacuated (building library))  
(emergency (emergency-time bomb) (location main-room))
```

Un problema di rappresentazione

I fatti non possono essere annidati

- Gli slot di un fatto non ordinato possono essere single-value o multi-value, ma ciascuno di questi valori deve essere VALUE (o un suo sotto-tipo)
- Come modellare, ad esempio, che una persona ha un indirizzo e l'indirizzo è a sua volta un fatto articolato?

```
(deftemplate person
  (slot name)
  (multislot address))
(person (name Franco)
       (address via Milano 12 Torino 10100 Italia))
```

I fatti non possono essere annidati

- Gli slot di un fatto non ordinato possono essere single-value o multi-value, ma ciascuno di questi valori deve essere VALUE (o un suo sotto-tipo)
- Come modellare, ad esempio, che una persona ha un indirizzo e l'indirizzo è a sua volta un fatto articolato?

```
(deftemplate person
  (slot name)
  (multislot address))
(person (name Franco)
        (address via Milano 12 Torino 10100 Italia))
```

- ma se poi cerco persone che abitano a Milano il pattern matching è più complicato (come distinguo via Milano da Milano città?)
- nascondo dentro un multislot un pezzo di informazione strutturata, perdo il significato di cosa siano Torino e Milano
- e se poi devo dire che ho una classe fatta di persone? metto tutto (nome e indirizzo) in un unico multislot?

I fatti non possono essere annidati

- Per legare logicamente due (o più) fatti occorre un attributo che li metta in relazione
- ad esempio un identificatore comune:

```
(deftemplate person
  (slot id)
  (slot name)
)
(deftemplate address
  (slot id)
  (slot street)
  (slot num)
  (slot zip-code)
  (slot city)
)
```

```
(person (id H725) (name Franco) )
(address (id H725) (street via Milano) (num 12)
         (city Torino) (zip-code 10100)))
```

I fatti non possono essere annidati

- Per legare logicamente due (o più) fatti occorre un attributo che li metta in relazione
- ad esempio un identificatore comune:

```
(deftemplate person
  (slot id)
  (slot name)
)
(deftemplate address
  (slot id)
  (slot street)
  (slot num)
  (slot zip-code)
  (slot city)
)
```

```
(person (id H725) (name Franco) )
(address (id H725) (street via Milano) (num 12)
         (city Torino) (zip-code 10100)))
```

- come generare identificatori automaticamente?

Funzioni gensym e gensym*

- (gensym)
restituisce un nuovo simbolo con forma `genX` dove `X` è un intero incrementato automaticamente ad ogni invocazione della funzione. Il primo simbolo generato è `gen1`. I simboli generati non sono necessariamente univoci
- (gensym*)
Simile a `gensym` ma con la garanzia che il simbolo generato è univoco
- (setgen X)
imposta il valore iniziale del numero in coda al simbolo

Esempi:

```
CLIPS> (setgen 1)
1
CLIPS> (assert (gen1 gen2 gen3))
<Fact-0>
CLIPS> (gensym)
gen1
CLIPS> (gensym*)
gen4
CLIPS>
```

Funzioni gensym e gensym* come default values

```
(deftemplate foo
  (slot w (default ?NONE)) ;richiede che ci sia sempre un valore
  (slot x)
  (slot y (default (gensym*))) ;il valore è determinato alla prima asserzione d
                                ;fatto foo
  (slot z (default-dynamic (gensym*))) ;il valore viene determinato ad
                                      ;ogni asserzione del fatto foo
```

```
CLIPS> (assert (foo))
```

```
[TMPLTRHS1] Slot w requires a value
because of its (default ?NONE) attribute.
```

```
CLIPS> (assert (foo (w 3)))
```

```
<Fact-0>
```

```
CLIPS> (assert (foo (w 4)))
```

```
<Fact-1>
```

```
CLIPS> (facts)
```

```
f-0      (foo (w 3) (x nil) (y gen1) (z gen2))
```

```
f-1      (foo (w 4) (x nil) (y gen1) (z gen3))
```

```
For a total of 2 facts.
```

```
CLIPS>
```

Modellare un mondo dinamico

- Sono utili per dare un identificatore ai fatti
- Possono essere usati anche per tracciare fatti distinti che però fanno riferimento collettivamente alla stessa informazione. Ad esempio, uno stato in A^* .
- Esempio: supponiamo di voler modellare il cambiamento di stato prodotto dall'azione *grasp* nel mondo dei blocchi



Come possiamo modellare lo stato prima e dopo l'azione?

Modellare un mondo dinamico

- Esempio: supponiamo di voler modellare il cambiamento di stato prodotto dall'azione *grasp* nel mondo dei blocchi



Potremmo pensare ad un template per modellare i blocchi e le loro posizioni rispetto agli altri blocchi

```
(deftemplate block
  (slot name)
  (slot on-top-of)
  (slot below-of)
)
```

Modellare un mondo dinamico

- Esempio: supponiamo di voler modellare il cambiamento di stato prodotto dall'azione `grasp` nel mondo dei blocchi



Potremmo pensare ad un template per modellare i blocchi e le loro posizioni rispetto agli altri blocchi

```
(deftemplate block
  (slot name)      (block (name A) (on-top-of TABLE) (below-of B))
  (slot on-top-of) (block (name B) (on-top-of A) (below-of NONE))
  (slot below-of)  (block (name C) (on-top-of TABLE) (below-of NONE))
)
```

Modellare un mondo dinamico

- Modelliamo una possibile azione grasp con una regola

```
(defrule grasp-on-top-of
  ?h <- (hand (holding NONE))
  ?b1 <- (block (name ?block) (on-top-of ?b1 & ~?block)
          (below NONE))
  ?b2 <- (block (name ?b1) (on-top-of ?x) (below-of ?block))
=>
  (retract ?h ?b1 ?b2)
  (assert (hand (holding ?block))
          (block (name ?block) (on-top-of GRASPED) (below-of NONE))
          (block (name ?b1) (on-top-of ?x) (below-of NONE)))
)
```

- Così facendo però mantengo solo l'ultimo stato, lo stato corrente, e se volessi tracciare una storia degli stati visitati?
- Tracciare la storia degli stati visitati è fondamentale per poter fare backtracking!

- Modifichiamo il template in modo da tracciare anche l'id di uno stato

```
(deftemplate block
  (slot state-id)
  (slot name)
  (slot on-top-of)
  (slot below-of)
)
```

```
(block (state-id gen1) (name A) (on-top-of TABLE) (below-of B))
(block (state-id gen1) (name B) (on-top-of A) (below-of NONE))
(block (state-id gen1) (name C) (on-top-of TABLE) (below-of NONE))
(hand (state-id gen1) (holding NONE))
```

Modellare un mondo dinamico

- Modifichiamo la regola che modella l'azione grasp

```
(defrule grasp-on-top-of
  (current-state-id ?id)
  ?h <- (hand (state-id ?id) (holding NONE))
  ?b1 <- (block (state-id ?id) (name ?block) (on-top-of ?b1 & ~?block)
          (below NONE))
  ?b2 <- (block (name ?b1) (on-top-of ?x) (below-of ?block))
=>
  (retract ?h ?b1 ?b2)
  (bind ?new (gensym*)) ;bind assegna valore a NUOVA variable nel conseguente
  (assert
    (next-state-id ?new)
    (hand (state-id ?new) (holding ?block) )
    (block (state-id ?new) (name ?block) (on-top-of GRASPED) (below-of NONE))
    (block (state-id ?new) (name ?b1) (on-top-of ?x) (below NONE))
  ) )
```

Modellare un mondo dinamico

- Modifichiamo la regola che modella l'azione grasp

```
(defrule grasp-on-top-of
  (current-state-id ?id)
  ?h <- (hand (state-id ?id) (holding NONE))
  ?b1 <- (block (state-id ?id) (name ?block) (on-top-of ?b1 & ~?block)
          (below NONE))
  ?b2 <- (block (name ?b1) (on-top-of ?x) (below-of ?block))
=>
  (retract ?h ?b1 ?b2)
  (bind ?new (gensym*)) ;bind assegna valore a NUOVA variable nel conseguente
  (assert
    (next-state-id ?new)
    (hand (state-id ?new) (holding ?block) )
    (block (state-id ?new) (name ?block) (on-top-of GRASPED) (below-of NONE))
    (block (state-id ?new) (name ?b1) (on-top-of ?x) (below NONE))
  ) )
```

- **Abbiamo un problema però:** La grasp mi dice solo cosa cambia: il nuovo stato contiene solo effetti dell'azione, ma tutto ciò che non è cambiato non compare nel nuovo stato: **Devo trattare esplicitamente il frame problem** con regole che replicano i fatti da uno stato al successivo se questi non sono cambiati

Modellare un mondo dinamico: Frame Problem

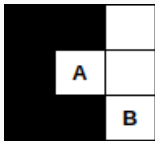
- tutto ciò che non è cambiato per effetto dell'azione deve persistere
- ogni blocco (e ogni altro elemento del dominio) sarà “ricopiato” nel nuovo stato

```
(defrule persistency-block (declare (salience 10))  
  (next-state-id ?new)  
  (current-state-id ?id)  
  ?b <- (block (state-id ?id) (name ?block))  
  (not (block (state-id ?new) (name ?block)))  
=>  
  (duplicate ?b (state-id ?new))  
)
```

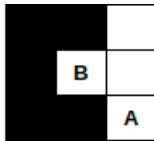
- dopo aver copiato tutti gli elementi persistenti, dovrò dire che il nuovo stato è quello corrente, notare quindi la salience

```
(defrule persistency-end (declare (salience 5))  
  ?n <- (next-state-id ?new)  
  ?o <- (current-state-id ?id)  
=>  
  (retract ?n ?o)  
  (assert (current-state-id ?new))  
)
```

Un caso di studio



Initial State



Goal State

- A e B sono due agenti che devono scambiarsi di posto
- ovviamente non possono occupare la stessa posizione nello stesso momento

Un caso di studio - Modellazione del dominio

0	1	2
3	4	5
6	7	8

- diamo un nome ad ogni cella
- modelliamo il layout come adiacenze tra le sole celle percorribili

```
(deffacts layout
  (neighbors (name 2) (left NONE) (right NONE) (up NONE) (down 5))
  (neighbors (name 4) (left NONE) (right 5) (up NONE) (down NONE))
  (neighbors (name 5) (left 4) (right NONE) (up 2) (down 8))
  (neighbors (name 8) (left NONE) (right NONE) (up 5) (down NONE))
)
```

- Ogni agente può muoversi in un cella adiacente purché sia libera

```
(defrule move-left-block
  ?initial <- (cell (name ?current) (content ?agent & ~empty & ~blocked))
  (neighbors (name ?current) (left ?move-to))
  ?final <- (cell (name ?move-to) (content empty))
=>
  (modify ?initial (content empty))
  (modify ?final (content ?agent))
)
```

- Tutto bene allora... o quasi
- Qual è il problema di questa soluzione?
- Come risolverlo?

Comando bind - esempi

- Lega esplicitamente un valore ad una variabile, può essere usato nel conseguente delle regole
- In particolare, nel conseguente delle regole possono essere usate anche “nuove” variabili, cioè non menzionate nell’antecedente

```
(bind ?distance (+ f g))
```

```
(bind ?new (gensym*))
```

```
(assert (car (state-id ?new) (name ?name) (position MI)))
```

```
(open <file-name> <logical-name> "r")
```

- <file-name> è il nome del file su disco
- <logical-name> è un nome usato all'interno del codice CLIPS
- "r" indica il permesso di lettura ("w" per scrivere)

```
(open "example.dat" my-file "r")
```

```
(read my-file)
```

- Al termine il file deve essere chiuso:

```
(close <logical-name>)
```

- `read` legge un solo simbolo
- `readline` legge un'intera riga terminata da CR

Struttura generale:

- `(read <logical-name>)` legge un solo token dal file
- `(read)` legge un solo token dal terminale (il file di default)

Esempi:

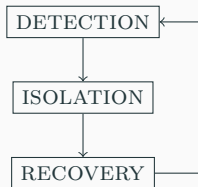
- `(bind ?input (read))`
- `(bind $?input (readline))`

Programmazione Modulare

- Programmi in domini reali coinvolgono facilmente migliaia di regole
- come abbiamo visto, non tutte le regole codificano *conoscenza di dominio* (cioè modellano il problema che si vuole risolvere), alcune regole codificano infatti *conoscenza di controllo*, che regola il comportamento del programma
- L'interleaving tra conoscenza di controllo e di dominio può rappresentare un serio problema nella manutenzione della KB nel tempo.

- Immaginiamo di dover risolvere un problema di diagnosi di un dispositivo
- l'analisi diagnostica si svolge in tre fasi successive:
 - Detection: riconoscere che esiste un guasto
 - Isolation: determinare quale componente del dispositivo ha causato il guasto
 - Recovery: determinare quali passi seguire per ripristinare la corretta funzione del dispositivo

Il processo può essere iterativo:



Prima possibile soluzione: Salience

- Per fare in modo che le regole della fase *recovery* vengano eseguite solo dopo che la fase di *isolation* sia terminata, e abbia seguito la fase di *detection*, si potrebbe ricorrere alla salience



- due possibili svantaggi:
 - la conoscenza di controllo è ancora distribuita tra le regole
 - non garantisce il corretto ordine dell'esecuzione

Prima possibile soluzione: Saliency - Esempio

```
(defrule fire-first (declare (saliency 30))  
  (priority first)
```

=>

```
(printout t "Print first" crlf))
```

```
(defrule fire-second (declare (saliency 20))  
  (priority second)
```

=>

```
(printout t "Print second" crlf))
```

```
(defrule fire-third (declare (saliency 10))  
  (priority third)
```

=>

```
(printout t "Print third" crlf))
```

- Se in WM sono presenti i fatti
(priority first), (priority second), (priority third)
l'agenda ordina le regole come ci si aspetta
- Ma se la WM contenesse solo: (priority second), (priority third)
allora regole della seconda fase sarebbero eseguite prima delle regola della prima fase

Fasi e Fatti di Controllo

- Una soluzione migliore per controllare il flusso dell'esecuzione è quella di separare *control knowledge* da *domain knowledge*
 - ad ogni regola è assegnato un **control pattern** che indica in quale fase quella regola è applicabile
 - regole di controllo sono poi definite per trasferire il controllo da una fase alla successiva

DOMAIN KNOWLEDGE



CONTROL KNOWLEDGE



- Regole di controllo

```
(defrule detection-to-isolation
  (declare (salience -10))
  ?phase <- (phase detection)
```

=>

```
  (retract ?phase)
  (assert (phase isolation)))
```

```
(defrule isolation-to-recovery
  (declare (salience -10))
  ?phase <- (phase isolation)
```

=>

```
  (retract ?phase)
  (assert (phase recovery)))
```

```
(defrule recovery-to-detection
  (declare (salience -10))
  ?phase <- (phase recovery)
```

=>

```
  (retract ?phase)
  (assert (phase detection)))
```

- Ogni regola che appartiene ad una fase specifica ha come primo pattern un fatto di controllo

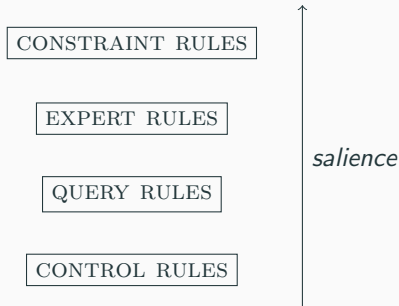
```
(defrule find-fault-location-and-recovery
  (phase recovery)
  (recovery-solution switch-device ?replacement on)
=>
  (printout t "Switch device" ?replacement "on" crlf) )
```

- In combinazione con la MEA questo pone tutte le regole attive della fase più recente in cima all'agenda

- Osservazione: mentre è attiva la fase di *detection* sarà anche attiva la regola di controllo *detection-to-isolation* (per il passaggio di fase da *detection* a *isolation*)
- La salience più bassa di questa regola garantisce che sarà eseguita solo dopo che le tutte le regole della fase di *detection* sono state eseguite



- In genere possiamo evidenziare quattro classi di knowledge che possono essere modellate in un sistema esperto



- Un possibile svantaggio:
 - Fatti di controllo partizionano logicamente la KB, ma la WM rimane unica
 - Regole in una fase $phase_1$ potrebbero accedere e modificare fatti che sono di esclusiva competenza di una fase successiva $phase_i$
 - Il problema è che con il crescere del programma non è sempre facile riconoscere queste situazioni, visto che il programma potrebbe essere scritto da persone diverse anche nell'arco di un periodo lungo (vedi problemi di manutenibilità)

- Uno svantaggio ancora più importante:
 - In certe situazioni vorrei poter passare da una fase all'altra in modo più flessibile
 - ad esempio, vorrei passare da *isolation* a *recover* per sistemare un particolare componente e poi tornare di nuovo alla fase di *isolation* e ritrovare tutti i fatti e le regole in agenda di prima
 - con i fatti di controllo questo non è possibile nel modo corretto perché richiede di rimuovere e riasserire il fatto (*phase isolation*)
 - ri-asserire (*phase isolation*) potrebbe abilitare molte più regole del voluto perché inibisce la rifrazione
- occorre quindi un modo per partizionare WM e KB più strutturato
⇒ i **moduli**

- CLIPS usa i moduli per partizionare KB e WM

```
(defmodule <module-name> [<comment>])
```
- In assenza di indicazioni, tutti i fatti, template e regole sono definite nel modulo MAIN
- volendo allora risolvere il problema della diagnosi di un dispositivo possiamo definire i tre moduli:

```
CLIPS> (defmodule DETECTION)
```

```
CLIPS> (defmodule ISOLATION)
```

```
CLIPS> (defmodule RECOVERY)
```

- da qui in avanti regole e template sono definite per default nell'ultimo modulo definito (che diventa quello corrente)
- ma si può specificare il modulo di destinazione al momento della definizione:

```
(defrule DETECTION::rule-detection-1 ... => ... )
```

```
(deftemplate ISOLATION::component (slot name) .... )
```

- NB i Moduli partizionano non solo la KB, ma anche la WM e di conseguenza anche l'agenda ⇒ **ogni modulo ha la propria agenda e WM**
- alcuni comandi utili per consultare WM/KB in presenza di moduli:
 - `(get-current-module)`
 - `(set-current-module module-name)`
 - `(list-defrules)` visualizza il contenuto del modulo corrente
 - `(list-defrules module-name)` visualizza le regole definite in *module-name* anche se non è il modulo corrente
 - `(list-defrules *)` visualizza le regole definite in tutti i moduli
 - il comportamento per `(facts)`, `(agenda)`, etc. è simile

- Ovviamente i moduli devono poter scambiare qualche informazione, altrimenti averli completamente partizionati non sarebbe di alcuna utilità
- i costrutti (`defrule`) e (`def facts`) sono strettamente privati, non possono cioè essere condivisi
- i costrutti (`def template`) possono essere definiti in un modulo ed importati in altri
- quando un (`def template`) è condiviso tutti i fatti non ordinati istanza di quel template sono anch'essi condivisi

- Esportare:

- `(defmodule module-name (export ?ALL))`
- `(defmodule module-name (export ?NONE))`
- `(defmodule module-name (export deftemplate ?ALL))`
- `(defmodule module-name (export deftemplate ?NONE))`
- `(defmodule module-name (export deftemplate
 <template-name>+))`

- Importare:

- `(defmodule module-name (import in-module ?ALL))`
- `(defmodule module-name (import in-module ?NONE))`
- `(defmodule module-name (import in-module deftemplate
 ?ALL))`
- `(defmodule module-name (import in-module deftemplate
 ?NONE))`
- `(defmodule module-name
 (import in-module deftemplate <template-name>+))`

- le regole che sono prese in considerazione per l'esecuzione sono quelle del modulo che in questo momento ha il *focus*, che non corrisponde all'ultimo modulo definito!
- i comandi di (reset) e (clear) portano il focus sul modulo MAIN
- L'esecuzione del programma parte quindi sempre dal modulo MAIN
- CLIPS gestisce il passaggio da un modulo ad un altro attraverso uno stack chiamato appunto **focus**

- con il comando (`focus module-name`) pone in cima allo stack focus il modulo indicato
- di conseguenza, tutte le regole definite in quel modulo saranno prese in esame ed eventualmente eseguite
- un modulo rimane in cima allo stack fino a quando
 - ci sono regole attive da eseguire, quando non ci sono più regole eseguibili, il modulo viene rimosso dalla cima dello stack (pop implicito)
 - una regola esegue il comando esplicito di pop: (`pop focus`) oppure (`return`)
 - una regola aggiunge un ulteriore modulo in cima allo stack (`focus altro-modulo`)

- è possibile specificare più moduli con il comando `focus`:
 `(focus DETECTION ISOLATION RECOVER)`
 il top è a sinistra
- il comando `(list-focus-stack)` visualizza il contenuto dello stack,
- in alternativa si può usare anche `(get-focus-stack)`
- anche il focus ha il suo watch: `(watch focus)`

- E' possibile specificare che alcune regole sono così importanti che, se le loro precondizioni diventano soddisfatte sono poste in cima all'agenda anche quando il modulo che ha il focus non è quello in cui sono definite
- un caso dove questo è molto utile è quando la regola riconosce la **violazione di un vincolo**

```
(defmodule CHECK (import MAIN ?ALL) (import FILM ?ALL))  
(defrule checks (declare (auto-focus TRUE))  
  ?x <- (ticket (name ?n) (title ?t))  
  (film (title ?) (VM YES))  
  (person (name ?n) (age ?a&:(< ?a 18)))  
=>  
  (retract ?x)  
  (printout t "No film for " ?n crlf)  
)
```

Sostituire i fatti di controllo con i moduli

```
(def facts MAIN::control-information
  (phase-sequence DETECTION ISOLATION RECOVERY))
```

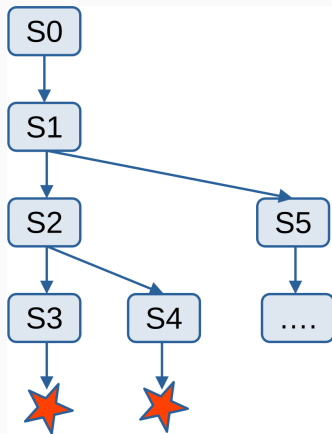
```
(defrule MAIN::change-phase
  ?list <- (phase-sequence ?next-phase $?other-phases)
=>
  (focus ?next-phase)
  (retract ?list)
  (assert (phase-sequence ?other-phases ?next-phase)))
```

- ogni volta che una fase termina il focus torna al modulo MAIN dove la regola change-phase sposta il focus sulla fase successiva
- confrontando questa soluzione con quella che fa ricorso ai fatti di controllo i vantaggi dovrebbero essere evidenti: molte meno regole, più facili da mantenere (es. invertire o aggiungere fasi)

Ricerca con Backtracking

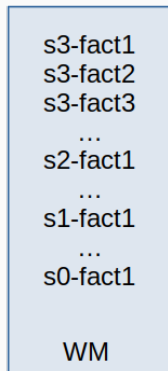
Problema: Come realizzare una strategia di ricerca in CLIPS?

- Per implementare una strategia di ricerca in profondità occorre:
 - mantenere un cammino aperto
 - per ogni stato lungo il cammino, l'insieme delle possibilità ancora da esplorare
 - saper fare backtracking



Sfruttiamo le strutture di CLIPS

- Per modellare un singolo stato usiamo un identificatore condiviso da tutti i fatti appartenenti a quello stato
- per modellare il cammino aperto usiamo la WM
- per tracciare le alternative non ancora esplorate usiamo l'agenda



- usando la strategia in profondità, le attivazioni delle regole sono stratificate in agenda in modo che in cima allo stack ci siano tutte e sole le regole applicabili allo stato più recente
- far scattare una regola vuol dire generare un nuovo stato successore
- le opzioni aperte non ancora esplorate rimangono in agenda
- quando non ci sono più regole applicabili per un dato stato s_i avviene un backtrackig (implicito) e si passa alla prossima regola in cima allo stack, che riguarderà uno stato s_j antenato di s_i

- Consideriamo il problema di far sedere N persone ad un tavolo in modo tale che:
 - ogni persona abbia accanto una persona del sesso opposto
 - ogni persona condivida almeno un hobby con i suoi vicini

- Consideriamo il problema di far sedere N persone ad un tavolo in modo tale che:
 - ogni persona abbia accanto una persona del sesso opposto
 - ogni persona condivida almeno un hobby con i suoi vicini
- Cominciamo con il modellare gli ospiti:
`(deftemplate guest(slot name)(slot sex)(slot hobby))`

- Ogni nodo dello spazio di ricerca corrisponde ad un assegnaemto di persone a certe sedie
- Più stati saranno aperti durante la ricerca
- Devo poter distinguere tra assegnamenti validi in uno stato da quelli validi in un altro stato
- occorre un identificatore dello stato:

```
(deftemplate path
  (slot id) ;accomuna i fatti appartenenti allo
stesso stato
  (slot name)
  (slot seat))
```

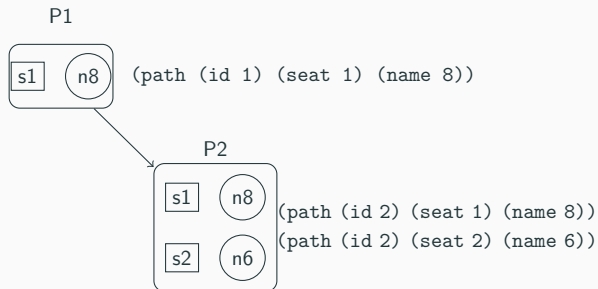
Modellare lo spazio di ricerca - Esempio

P1

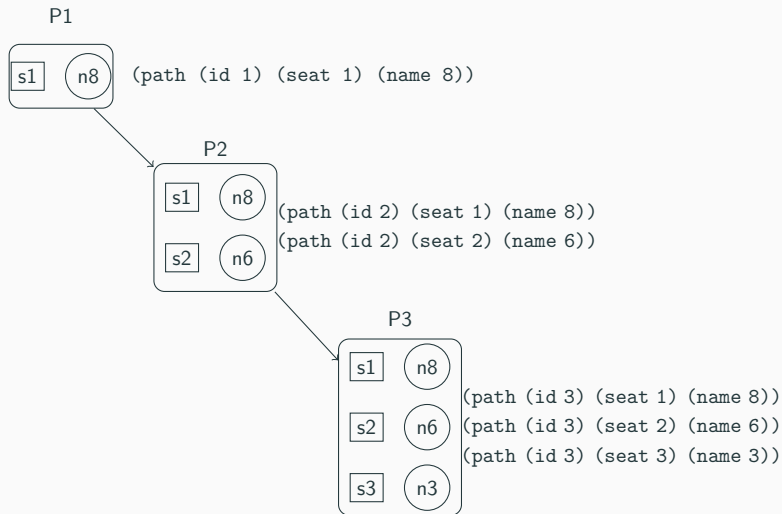


(path (id 1) (seat 1) (name 8))

Modellare lo spazio di ricerca - Esempio



Modellare lo spazio di ricerca - Esempio



- Altra struttura di supporto per la definizione dello stato corrente
- coppie di assegnamenti consecutivi:

```
(deftemplate seating
  (slot seat1) (slot name1) (slot name2) (slot
seat2)
  (slot id) (slot pid) (slot path_done))
```

- ogni assegnamento ha un suo id, ma appartiene ad un path (pid):

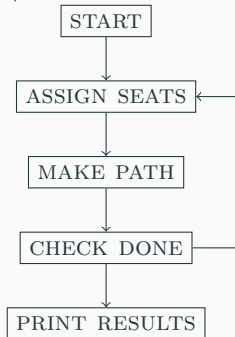
```
seating (seat1 1) (name1 n8) (seat2 2) (name2 n8) (id 1) (pid 0)
(path_done yes))
seating (seat1 1) (name1 n8) (seat2 2) (name2 n8) (id 2) (pid 1)
(path_done yes))
seating (seat1 1) (name1 n8) (seat2 2) (name2 n8) (id 3) (pid 2)
(path_done yes))
```

- gli ospiti e i loro hobby

```
(deftemplate guest (slot name) (slot sex) (slot
hobby) )
```

```
(guest (name n1) (sex m) (hobby h3)) (guest (name n1) (sex m)
(hobby h2))
(guest (name n2) (sex m) (hobby h2)) (guest (name n2) (sex m)
(hobby h3))
(guest (name n3) (sex m) (hobby h1)) (guest (name n3) (sex m)
(hobby h2)) (guest (name n3) (sex m) (hobby h3))
(guest (name n4) (sex f) (hobby h3)) (guest (name n4) (sex f)
(hobby h2))
(guest (name n5) (sex f) (hobby h1)) (guest (name n5) (sex f)
(hobby h2)) (guest (name n5) (sex f) (hobby h3))
(guest (name n6) (sex f) (hobby h3)) (guest (name n6) (sex f)
(hobby h1)) (guest (name n6) (sex f) (hobby h2))
(guest (name n7) (sex f) (hobby h3)) (guest (name n7) (sex f)
(hobby h2)) (guest (name n8) (sex m) (hobby h3)) (guest (name n8)
(sex m) (hobby h1))
```

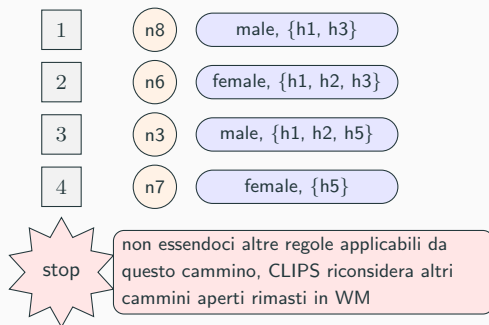
- Organizzata per contesti/fasi



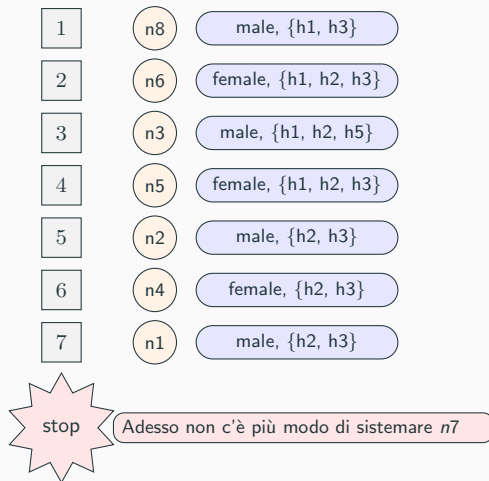
Esempio di esecuzione senza backtracking

1	n8	male, {h1, h3}
2	n6	female, {h1, h2, h3}
3	n3	male, {h1, h2, h3}
4	n7	female, {h2, h3}
5	n2	male, {h2, h3}
6	n5	female, {h1, h2, h3}
7	n1	male, {h2, h3}
8	n4	female, {h2, h3}

Esempio di esecuzione con backtracking



Esempio di esecuzione con backtracking



Esempio di esecuzione con backtracking

1	n8	male, {h1, h3}
2	n6	female, {h1, h2, h3}
3	n2	male, {h2, h3}
4	n5	female, {h1, h2, h3}
5	n1	male, {h2, h3}
6	n4	female, {h2, h3}
7	n3	male, {h1, h2, h5}
8	n7	female, {h5}

- Dopo molti backtracking, verrà esplorata la soluzione in cui $n3$, l'unico compatibile con $n7$, siederà in penultima posizione.

Incertezza - Certainty Factors

- CLIPS non ha una funzionalità built-in per la gestione del ragionamento incerto
- E' possibile però incorporare incertezza usando una strategia simile a quella usata in MYCIN

Esempio di regola MYCIN

```
IF (1) The stain of the organism is gramneg and  
   (2) The morphology of the organism is rod and  
   (3) The patient is a compromised host  
THEN There is suggestive evidence (0.6) that the identity  
of the organism is pseudoinonas
```

- Intuitivamente la regola equivale ad una probabilità condizionata

$$P(H|E_1 \cap E_2 \cap E_3) = 0.6$$

Certainty Factors $\neq$ Probabilità

- MYCIN opera nel contesto medico
- I medici non ragionano in termini probabilistici, ma per gradi di *belief* e *disbelief*
- questi gradi di belief non sono necessariamente consistenti in termini probailistici!
- cioè il medico non ritiene necessariamente vera l'equazione

$$P(not.H|E_1 \cap E_2 \cap E_3) = 1 - 0.6$$

dove *not.H* è un'ipotesi complementare ad *H*.

- La ragione è che:

Certainty Factors $\neq$ Probabilità

- MYCIN opera nel contesto medico
- I medici non ragionano in termini probabilistici, ma per gradi di *belief* e *disbelief*
- questi gradi di belief non sono necessariamente consistenti in termini probabilistici!
- cioè il medico non ritiene necessariamente vera l'equazione

$$P(not.H|E_1 \cap E_2 \cap E_3) = 1 - 0.6$$

dove *not.H* è un'ipotesi complementare ad *H*.

- La ragione è che:
 - mentre è vero che E_1, E_2, E_3 hanno una relazione causa-effetto con H , da cui segue $P(H|E_1 \cap E_2 \cap E_3)$
 - non è necessariamente vero che E_1, E_2, E_3 hanno una relazione causa-effetto con *not.H*, quindi $P(not.H|E_1 \cap E_2 \cap E_3)$ è corretta probabilisticamente, ma non è giustificata nella pratica

Certainty Factors $\neq$ Probabilità

- E_1, E_2, E_3 devono essere intese come evidenze a favore di H senza però concludere nulla circa $not.H$
- Le evidenze non indicano che l'ipotesi è confermata, ma solo che si è portati a crederci con un certo “peso qualitativo”
- I CF permettono una forma di ragionamento inesatto quando forme di ragionamento (probabilistico) esatto sono impraticabili per mancanza di conoscenza

Certainty Factors: Definizione

- Certainty Factors (Carnap 1950) come alternativa alle probabilità per catturare un grado di conferma (*degree of confirmation* o *epistemic probability*)
- Un CF conferma un'ipotesi data una qualche evidenza
- In MYCIN un CF è stata definita come la differenza tra belief e disbelief:

$$CF(H, E) = MB(H, E) - MD(H, E)$$

dove

- CF è il fattore di certezza nell'ipotesi H data l'evidenza E
- MB è la **misura di crescita della credenza** (belief) in H dato E
- MD è la **misura di riduzione della credenza** in H data E

Certainty Factors: Definizione

- Certainty Factors (Carnap 1950) come alternativa alle probabilità per catturare un grado di conferma (*degree of confirmation* o *epistemic probability*)
- Un CF conferma un'ipotesi data una qualche evidenza
- In MYCIN un CF è stata definita come la differenza tra belief e disbelief:

$$CF(H, E) = MB(H, E) - MD(H, E)$$

dove

- CF è il fattore di certezza nell'ipotesi H data l'evidenza E
- MB è la **misura di crescita della credenza** (belief) in H dato E
- MD è la **misura di riduzione della credenza** in H data E
- Quindi dire che $CF(H, E) = .7$ vuol dire che il grado di **credenza a favore** dell'ipotesi cresce di .7 con l'evidenza E .
- viceversa $CF(H, E) = -.7$ vuol dire che il grado di **credenza a sfavore** di H cresce di .7 con l'evidenza E

Certainty Factor

rappresenta il grado "netto" di credenza sulla verità di un'ipotesi basata su una qualche evidenza. E' un valore nell'intervallo $[-1, 0, +1]$.

- 1 il fatto è noto essere vero per certo
 - 0 non ci sono abbastanza elementi per affermare che il fatto sia vero o falso (incertezza massima)
 - -1 il fatto è noto essere falso
-
- **Un CF combina quindi belief e disbelief in un unico valore**
 - Questo comporta almeno due vantaggi:
 - Possibile ragionare in assenza di informazione completa
 - Ranking delle ipotesi disbelief.

Come simulare MYCIN in CLIPS: OAV

- MYCIN rappresenta fatti come una tripletta *object-attribute-value* (OAV)
- Questa trippletta può essere modellata usando il seguente template

```
(defmodule OAV (export deftemplate oav))
(deftemplate OAV::oav
  (multislot object (type SYMBOL))
  (multislot attribute (type SYMBOL))
  (multislot value)
  (slot CF (type FLOAT) (range -1.0 +1.0)))
```

Modellare Conoscenza da MYCIN a CLIPS (esempio)

```
(oav (object organism)
      (attribute stain)
      (value gramneg)
      (CF 0.3))
```

```
(oav (object organism)
      (attribute morphology)
      (value rod)
      (CF 0.7))
```

```
(oav (object patient)
      (attribute is a)
      (value compromised host)
      (CF 0.8))
```

- MYCIN consente di derivare la stessa tripla OAV da path di ragionamento distinti (i.e., da diverse concatenazioni di regole) e quindi combinare i corrispondenti fattori di certezza
- CLIPS per default non consente di avere fatti identici
- due OAV con CF diversi sono ammessi, ma non due OAV con medesimo CF

Simulare il ragionamento di MYCIN

- MYCIN consente di derivare la stessa tripla OAV da path di ragionamento distinti (i.e., da diverse concatenazioni di regole) e quindi combinare i corrispondenti fattori di certezza
- CLIPS per default non consente di avere fatti identici
- due OAV con CF diversi sono ammessi, ma non due OAV con medesimo CF
- E' possibile disabilitare questa restrizione:
(set-fact-duplication TRUE)

Simulare il ragionamento di MYCIN

- MYCIN consente di derivare la stessa tripla OAV da path di ragionamento distinti (i.e., da diverse concatenazioni di regole) e quindi combinare i corrispondenti fattori di certezza
- CLIPS per default non consente di avere fatti identici
- due OAV con CF diversi sono ammessi, ma non due OAV con medesimo CF
- E' possibile disabilitare questa restrizione:
(set-fact-duplication TRUE)
- Avendo due fatti relativi allo stesso aspetto del problema si pone il problema di come combinarne i CF?

Simulare il ragionamento di MYCIN

- Dati due OAV identici nei loro primi tre slot
 - oav_1 con CF_1
 - oav_2 con CF_2
- Si ottiene un nuovo oav con

$$NewCertainty = (CF_1 + CF_2) - (CF_1 * CF_2)$$

- Esempio

(oav (object organism)	(oav (object organism)
(attribute morphology)	(attribute morphology)
(value rod)	(value rod)
(CF 0.7))	(CF 0.5))

$$New\ Certainty = (0.7 + 0.5) - (0.7 * 0.5) = 0.85$$

Implementativamente

```
(defrule OAV::combine-certainties-both-positive
  (declare (auto-focus TRUE)
    ?fact1 <- (oav (object $?o)
      (attribute $?a)
      (value $?v)
      (CF ?C1&:(>= ?C1 0)))
    ?fact2 <- (oav (object $?o)
      (attribute $?a)
      (value $?v)
      (CF ?C2&:(>= ?C2 0)))
    (test (neq ?fact1 ?fact2))
  =>
    (retract ?fact1)
    (bind ?C3 (- (+ ?C1 ?C2) (* ?C1 ?C2)))
    (modify ?fact2 (CF ?C3)))
```

- L'auto-focus garantisce che due oav identici saranno uniti prima di essere utilizzati in altre regole

E quando i CF non sono entrambi positivi...

- Oltre alla precedente regola vanno anche aggiunte altre due regole che considerano tutte le possibili combinazioni dei segni
- Se CF_1 e CF_2 sono entrambi negativi

$$\text{New Certainty} = (CF_1 + CF_2) + (CF_1 * CF_2)$$

- Se hanno segno opposto e quindi $-1 \leq CF_1 * CF_2 < 0$

$$\text{New Certainty} = \frac{CF_1 + CF_2}{1 - \min\{|CF_1|, |CF_2|\}}$$

Non basta! Come combino i CF dei fatti nelle attivazioni?

- Una regola è attivata da fatti (LHS), ciascuno con il proprio CF
- La regola stessa potrebbe essere associata ad un CF (in MYCIN ogni asserzione ha un CF “a priori”)

Non basta! Come combino i CF dei fatti nelle attivazioni?

- Una regola è attivata da fatti (LHS), ciascuno con il proprio CF
- La regola stessa potrebbe essere associata ad un CF (in MYCIN ogni asserzione ha un CF “a priori”)
- allo stesso tempo, la regola attivata produrrà nuovi fatti (RHS), con quale CF?
- Come combinare i CF delle attivazioni con i CF delle regole (i.e., asserzioni) per assegnare un CF alle conclusioni (cioè i fatti asseriti dalle regole)?

Combinare i CF delle attivazioni (LHS)

- A seconda del conditional element usato nell'antecedente possiamo avere:
 - $CF(P1 \text{ or } P2) = \max\{CF(P1), CF(P2)\}$
 - $CF(P1 \text{ and } P2) = \min\{CF(P1), CF(P2)\}$
 - $CF(\text{not } P) = -CF(P)$

Dove $P1$ e $P2$ sono pattern che sono soddisfatti da fatti

- MYCIN aggiunge un ulteriore vincolo: se il CF risultante è inferiore a 0.2, la regola viene considerata inapplicabile (il razionale è che la regola è troppo poco probabile)

Ottenere i CF dei fatti asseriti (RHS)

- Il CF dei fatti prodotti è determinato **moltiplicando** il CF dell'asserzione con il CF dell'attivazione dell'antecedente

Esempio di regola MYCIN

```
IF (1) The stain of the organism is gramneg and  
    (2) The morphology of the organism is rod and  
    (3) The patient is a compromised host  
THEN There is suggestive evidence (0.6) that the identity  
of the organism is pseudoinonas
```

- Qui 0.6 è il CF dell'asserzione, ma è espressa in termini assoluti, senza cioè considerare il grado di certezza sui fatti che attivano la regola.

In CLIPS...

```
(defmodule IDENTIFY (import OAV deftemplate oav))
(defrule IDENTIFY::MYCIN-to-CLIPS-translation
  (oav (object organism)
    (attribute stain)
    (value gramneg)
    (CF ?C1))
  (oav (object organism)
    (attribute morphology)
    (value rod)
    (CF ?C2))
  (oav (object patient)
    (attribute is a)
    (value compromised host)
    (CF ?C3))
  (test (> (min ?C1 ?C2 ?C3) 0.2))
=>
  (bind ?C4 (* (min ?C1 ?C2 ?C3) 0.6))
  (assert (oav (object organism)
    (attribute identity)
    (value pseudomonas)
    (CF ?C4))))
```

Un esempio di uso dei CF: WINE

Wine: Come scegliere il vino

- Obiettivo: determinare con un certo CF quale vino scegliere sulla base dei gusti dell'utente e del cibo che andrà servito
- il programma è organizzato nei seguenti moduli
 - QUESTIONS → RULES
 - CHOOSE-QUALITIES
 - WINES
 - PRINT-RESULTS

Modellare l'expertise

- Le regole di dominio sono del tipo:

if tastiness is average

then best-body is light with certainty 30 and

best-body is medium with certainty 60 and

best-body is full with certainty 30

- i CF sono nell'intervallo $[0,100]$
- notare che

- la clausola *if* ha sempre il formato

if *<attribute>* is *<value>*

- la clausola *then* ha sempre il formato

then *<attribute>* is *<value>* with certainty XX

questo semplifica la parsificazione della regola (vedi seguito)

Modellare l'expertise

- Le regole di dominio sono nettamente separate dalle regole di controllo
- Sono definite da fatti non ordinati

```
(deftemplate RULES::rule
(slot certainty (default 100.0))
(multislot if)
(multislot then))
```

Ad esempio

```
(rule (if tastiness is average)
      (then best-body is light with certainty 30 and
            best-body is medium with certainty 60 and
            best-body is full with certainty 30))
```

- notare che la regola (asserzione) ha CF 100, mentre le singole conseguenze hanno diversi CF, la cui somma non è 100 pur essendo mutuamente esclusivi!

- Gli antecedenti delle regole possono anche includere congiunzioni

Ad esempio

```
(rule (if main-component is poultry and has-turkey is no)
      (then best-color is white with certainty 90 and
            best-color is red with certainty 30))
```

- Definisce le (meta-)regole per porre ed interpretare domande
- le domande, come le regole, sono definite come fatti non ordinati

```
(deftemplate QUESTIONS::question
  (slot attribute (default ?NONE))
  (slot the-question (default ?NONE))
  (multislot valid-answers (default ?NONE))
  (slot already-asked (default FALSE))
  (multislot precursors (default ?DERIVE)))
```

II modulo QUESTIONS

- Esempi di questions

```
(question (attribute main-component)
  (the-question "Is the main component of the meal meat,
fish, or poultry? ")
  (valid-answers meat fish poultry unknown))}

(question (attribute has-turkey)
  (precursors main-component is poultry)
  (the-question "Does the meal have turkey in it? ")
  (valid-answers yes no unknown))
```

Regole per domande

```
(defrule QUESTIONS::ask-a-question
  ?f <- (question (already-asked FALSE)
    (precursors)
    (the-question ?the-question)
    (attribute ?the-attribute)
    (valid-answers $?valid-answers))
=>
  (modify ?f (already-asked TRUE))
  (assert (attribute (name ?the-attribute)
    (value (ask-question ?the-question ?valid-answers)))))
```

- (ask-question) è una funzione che visualizza la domanda, legge la risposta (verificandone la correttezza) e la assegna a value.

```
(deffunction MAIN::ask-question (?question ?allowed-values)
  (printout t ?question)
  (bind ?answer (read))
  (if (lexemep ?answer) then (bind ?answer (lowercase ?answer)))
  (while (not (member ?answer ?allowed-values)) do
    (printout t ?question)
    (bind ?answer (read))
    (if (lexemep ?answer) then (bind ?answer (lowercase ?answer))))
  ?answer)
```

- Definisce le (meta-)regole per interpretare (e semplificare) le regole di dominio

```
(defrule RULES::remove-is-condition-when-satisfied
  ?f <- (rule (certainty ?c1)
              (if ?attribute is ?value $?rest))
  (attribute (name ?attribute)
              (value ?value)
              (certainty ?c2))

=>
  (modify ?f (certainty (min ?c1 ?c2)) (if ?rest)))
```

- In passaggi successivi il CF dell'antecedente accumula la certezza data dalla combinazione di input dell'utente

- Quando l'antecedente di una regola si svuota ...

```
(defrule RULES::perform-rule-consequent-with-certainty
  ?f <- (rule (certainty ?c1)
    (if)
    (then ?attribute is ?value with certainty ?c2 $?rest))
=>
  (modify ?f (then ?rest))
  (assert (attribute (name ?attribute)
    (value ?value)
    (certainty (/ (* ?c1 ?c2) 100)))))
```

... asseriamo un attributo con un determinato CF che combina il CF accumulato dalla regola con quello associato “a priori”

- il database dei vini con le loro caratteristiche:

```
(wine (name Gamay) (color red) (body medium)
      (sweetness medium sweet))
```

```
(wine (name Chablis) (color white) (body light)
      (sweetness dry))
```

```
(wine (name Sauvignon-Blanc) (color white) (body medium)
      (sweetness dry))
```

```
(wine (name Chardonnay) (color white) (body medium full)
      (sweetness medium dry))
```

...

Notare che possono esistere più varianti dello stesso vino (es. Chardonnay può variare nel corpo e nella dolcezza)

- la regola per generare le proposte:

```
(defrule WINES::generate-wines
  (wine (name ?name)
        (color $? ?c $?)
        (body  $? ?b $?)
        (sweetness $? ?s $?))
  (attribute (name best-color) (value ?c) (certainty ?certainty-1))
  (attribute (name best-body) (value ?b) (certainty ?certainty-2))
  (attribute (name best-sweetness) (value ?s) (certainty ?certainty-3))
=>
  (assert (attribute (name wine) (value ?name)
                    (certainty (min ?certainty-1 ?certainty-2 ?certainty-3)))))
```

Se esiste un vino che soddisfa le caratteristiche (attribute) determinate dalle preferenze dell'utente, quel vino è suggerito con CF pari al minimo dei CF degli attributi

Regola generale per la combinazione dei CF

- nel caso lo stesso attributo venisse generato da più regole

```
(defrule MAIN::combine-certainties ""  
  (declare (salience 100)  
    (auto-focus TRUE))  
  ?rem1 <- (attribute (name ?rel) (value ?val) (certainty ?per1))  
  ?rem2 <- (attribute (name ?rel) (value ?val) (certainty ?per2))  
  (test (neq ?rem1 ?rem2))  
=>  
  (retract ?rem1)  
  (modify ?rem2  
    (certainty (/ (- (* 100 (+ ?per1 ?per2))  
      (* ?per1 ?per2)) 100))))
```